Controlled State-Dimensionality Study for Weekly Threshold Control in Credit Scoring

Contents

1 Introduction **3**

2 Problem Framing and Research Motivation **4**
- 2.1 Why weekly threshold control is the research object . . . 4
- 2.2 Why delayed outcomes and delayed rewards matter . . . 4
- 2.3 Why separate thresholds for new and repeat clients are essential . . . 5

3 Reference Boundary and Current Repository Scope **5**

4 Environment and Portfolio Simulator **6**
- 4.1 Episode timeline . . . 6
- 4.2 Weekly simulation logic . . . 6
- 4.3 Scenario design . . . 7

5 Controlled Experimental Design **8**
- 5.1 What is held fixed . . . 8
- 5.2 What changes . . . 8
- 5.3 Observation layers . . . 8

6 Controllers **9**
- 6.1 Rule-based and static baselines . . . 9
- 6.2 RL agents . . . 10

7 Reward Formulation and Metric Design **10**
- 7.1 Reward function . . . 10
- 7.2 Why this trade-off matters . . . 11
- 7.3 Reported metrics . . . 11

8 Experimental Protocol **12**

9 Main Results **13**
- 9.1 Best overall controller by dimension . . . 13
- 9.2 Best RL controller by dimension . . . 13
- 9.3 Algorithm-specific interaction with dimensionality . . . 14

9.4 Cross-dimension figures . . . 15

10 Scenario-Level Results and Interpretation **17**
10.1 Why scenario-level analysis is necessary . . . 17
10.2 Scenario trade-offs in the richest state . . . 17
10.3 Adverse stress . . . 18
10.4 Base market . . . 18
10.5 Class-imbalance shift . . . 19
10.6 Drift . . . 19
10.7 Noise . . . 19
10.8 Split-policy dynamics . . . 20

11 Threshold Behavior and Dynamic Response **20**
11.1 What threshold volatility actually shows . . . 20
11.2 Episode-static RL winners . . . 20
11.3 Early-loss / late-gain diagnostic . . . 21

12 Figures for Dimension Comparison **22**

13 Discussion **23**
13.1 Why 30D is the most balanced state size . . . 23
13.2 Why 50D saturates . . . 23
13.3 Why the best baseline still wins overall . . . 23
13.4 Portfolio-quality trade-offs . . . 24
13.5 What the current results do and do not prove . . . 24

14 Limitations **24**

15 Novelty and Contribution **25**

16 Conclusion **25**

1 Introduction

This report describes the current state of the repository after it was re-centered around a controlled state-dimensionality experiment for weekly threshold control in credit scoring. The project is not a generic benchmark on public tabular datasets and it is not a pointwise approval classifier. Its object of control is a portfolio policy. Each week the controller selects score cutoffs for new and repeat applicants, the simulator accepts all applications above those cutoffs, and the resulting cohort produces repayments, defaults, and late recoveries over future weeks. Because the timing of cash flows is delayed, portfolio quality cannot be evaluated from the current cohort alone. A policy that looks attractive on a one-week horizon can still produce a weaker end-of-episode portfolio once defaults and capital pressure materialize.

This distinction is the main reason the repository remains relevant as a research artifact. In real consumer lending, threshold design is often revised at a batch level rather than on every individual file. The operational question is therefore not only "which applications are good today," but also "which weekly score frontier keeps the portfolio profitable, solvent, and stable over time." The present simulator makes that temporal trade-off explicit. It models weekly application inflow, segment heterogeneity, delayed loss realization, partial recoveries, portfolio-level capital usage, and a reward function that combines profitability with risk and operational constraints.

The current repository focus is narrower and more controlled than the broader framework from which the project evolved. The central research question is now: *how much does observation design matter when everything else in the threshold-control problem is held fixed?* To answer this, the implementation compares four observation sizes, namely 12, 20, 30, and 50 features, while keeping environment dynamics, reward design, warm-up logic, terminal settlement, seeds, scenarios, baselines, RL algorithms, and the train/evaluation protocol unchanged.

That controlled setup matters for interpretation. It allows the dimensionality result to be attributed to the information content and complexity of the state vector rather than to hidden changes in reward shaping, scenario difficulty, or comparison controllers. The main exported conclusion is not that reinforcement learning automatically dominates rule-based policies. In fact, the strongest overall controller in the quick-profile outputs remains a non-RL baseline. The more precise conclusion is that state design materially changes RL quality, that richer portfolio state helps up to a point, and that the gain is strongly algorithm-dependent.

The updated repository supports a richer thesis-style narrative than the previous compact note. The main contributions evidenced by the current code and outputs are the following.

- The simulator implements weekly portfolio control with delayed credit outcomes, delayed rewards, terminal settlement, and separate thresholds for new and repeat clients.
- The dimensionality experiment changes only `state_dim`, which makes the 12D–20D–30D–50D comparison a controlled methodological study rather than a loose sequence of experiments.
- The exported tables show that the best RL frontier improves from 12D to 30D and then largely saturates from 30D to 50D.
- The same exports also show that additional dimensionality does not help all algorithms equally: DQN improves substantially, Double-DQN peaks earlier and collapses at 50D, PPO is effectively unchanged, and SAC remains unstable.

- Weekly reward maximization and raw profit maximization are not identical objectives in this repository. The results therefore reveal trade-offs among profitability, default rate, approval rate, capital usage, and volatility rather than a single scalar ranking only.

2 Problem Framing and Research Motivation

2.1 Why weekly threshold control is the research object

Many credit-scoring discussions are framed as one-shot classification problems: estimate default probability, choose a threshold, and accept or reject. That view is useful for static offline validation, but it misses the managerial reality that threshold decisions are often implemented at a portfolio level and revised periodically. A lender does not only care about cross-sectional predictive accuracy. It also cares about how this week's approval policy changes next month's outstanding balance, the realized default rate in the reward window, the available capital headroom, and the composition of new versus repeat borrowers.

The repository therefore treats the control variable as a weekly threshold policy. The agent does not see one application at a time and it does not make one-step approval decisions. Instead, it controls a score frontier for the next weekly batch. This is closer to a practical policy-management problem: the model must choose a weekly level of selectivity under uncertainty, then observe delayed consequences.

This framing has two academic advantages. First, it turns the problem into a genuine sequential decision process with delayed consequences. Second, it makes policy evaluation economically interpretable. A weekly threshold can be discussed in terms of profitability, capital usage, and realized defaults in a way that resembles portfolio management rather than abstract classification.

2.2 Why delayed outcomes and delayed rewards matter

The simulator explicitly separates origination from realization. When a loan is accepted in week t, the relevant cash flow does not necessarily arrive in week t. A performing loan produces payoff only after its duration elapses. A default creates a negative cash flow, and a recovered default creates a later positive recovery cash flow. As a result, the immediate quality of the current cohort is not a sufficient performance measure for the controller.

This is not a cosmetic modeling choice. It directly changes what "good control" means. A policy may accept a slightly riskier cohort today and appear to increase current approval or current expected profit, but after the delayed defaults arrive the total portfolio path can be worse. Conversely, a stricter policy can look locally conservative and temporarily underperform, yet generate a better end-of-horizon trajectory because later defaults are avoided and capital is preserved.

The repository keeps this logic in two different places. First, delayed outcomes are created in the simulator itself: accepted loans schedule future repayment, default-loss, and optional late-recovery events. Second, delayed reward is enabled in the active configuration, which means that the reward function uses realized portfolio cash flows instead of only the current cohort's expected profit. This is why the thesis interpretation must focus on full-trajectory portfolio quality rather than one-step cohort economics.

The diagnostic artifact `outputs/dim_30/locally_worse_globally_better.csv` makes the idea concrete. In the adverse-stress setting, several agents exhibit a negative early gap but a large positive final gap, meaning that short-run underperformance and long-run portfolio superiority can coexist. For example, the 30D DQN row records an early gap of approximately -643.46 and a final gap of approximately 250,578.20 under the pipeline's diagnostic comparator. This does not prove that RL always wins under stress. It does show why delayed evaluation is essential: early and final rankings are not guaranteed to coincide.

2.3 Why separate thresholds for new and repeat clients are essential

The simulator distinguishes new and repeat applicants at the structural level. Their base score distributions, default-response functions, recovery probabilities, loan sizes, and durations differ. This distinction is economically plausible: repeat clients typically contain more behavioral information and often exhibit different risk-return characteristics from newly acquired borrowers.

Because of that structural asymmetry, a shared threshold can be suboptimal even when both segments are scored on the same nominal scale. The repository therefore treats split-policy control as the default design. The action is normally a pair of thresholds rather than a single cutoff, and the observation includes segmented approval and default indicators. The dedicated `split_policy_dynamics` scenario strengthens this point by making new clients deteriorate while repeat clients improve.

This split-policy logic is especially important for interpretation of dimensionality. A richer state is useful only if the additional coordinates capture economically meaningful asymmetry. Several of the 20D, 30D, and 50D additions do exactly that: they add segment shares, segment-specific expected default indicators, accepted-share ratios, and threshold-gap history. The experiment is therefore not about abstract dimensionality for its own sake, but about whether a richer description of segmented weekly portfolio state helps the controller.

3 Reference Boundary and Current Repository Scope

The folder `Optimizing-Acceptance-Threshold-in-Credit-Scoring-using-Reinforcement-Learnin` remains a reference-only snapshot and is not the source of truth for the current report. According to `notes/reference_study.md`, the following mechanics were retained from the earlier study: weekly interaction, threshold control, delayed outcomes, delayed rewards, warm-up logic, and scenario-based distribution shift. The active implementation extends that foundation in several directions that are essential for the present thesis interpretation.

- Split-policy control for new and repeat applicants is part of the main design rather than an edge case.
- The experiment runner is configuration-driven and supports a controlled comparison of observation sizes.
- Reward shaping includes explicit portfolio-level penalties for default rate, approval pressure, capital usage, and volatility.

- The controller set is broader: DQN, Double-DQN, PPO, and SAC are compared against several non-RL baselines.
- Evaluation is multi-scenario, multi-seed, and exported with bootstrap confidence intervals and figure-ready artifacts.

This boundary is important because the thesis text should not describe the project as if it still used the original one-dimensional state or a single shared threshold only. The current repository is a more elaborate simulation framework whose immediate empirical focus is the controlled dimensionality study.

4 Environment and Portfolio Simulator

4.1 Episode timeline

The current environment follows a three-stage timeline that is implemented explicitly in `src/rl_credit_scoring_sim/env/simulator.py`.

1. **Warm-up phase.** The simulator runs 8 initial weeks with the default thresholds. These weeks populate outstanding loans, recent profit history, and rolling default information before the controller begins to act.
2. **Interactive phase.** During the 26 interactive weeks of the quick profile, the controller chooses weekly thresholds and the simulator records the resulting approvals, expected cohort characteristics, realized portfolio cash flows, and reward components.
3. **Terminal settlement.** When the interactive horizon ends, remaining pending interactive cash-flow events are settled so that the episode summary is not artificially truncated by loans whose repayment or recovery would have occurred after the last decision week.

This terminal-settlement step is especially important. Without it, a controller that originates many longer-duration loans near the end of the episode would be penalized mechanically because their positive cash flows would fall outside the measured window. The current implementation adds both terminal profit and terminal NPV to the final week's realized values, which makes the episode summary more faithful to the actual portfolio consequences of the policy.

4.2 Weekly simulation logic

Each simulated week begins with a scenario-specific market state. The scenario defines how repeat share, score drift, default drift, noise level, recovery probability, loan amount multiplier, and shock timing evolve over the horizon. Given that market state, the simulator draws weekly volumes for new and repeat segments, generates score distributions for both segments, converts scores into default probabilities through segment-specific logistic response functions, samples repayment/default/recovery outcomes, and assigns loan principal and duration distributions.

The controller then supplies a pair of thresholds $(\tau_t^{new}, \tau_t^{repeat})$. Every application with score above the relevant threshold is accepted. Accepted loans contribute immediately to the future event queue, not necessarily to current realized profit. The simulator records both expected current-cohort quantities and realized portfolio quantities. This separation matters:

- `expected_profit_current` and `expected_npv_current` summarize the newly accepted cohort under the current week's market conditions;
- `realized_profit` and `realized_npv` summarize the cash flows that actually arrive in the current week from loans originated earlier.

The episode summary exported to the main CSV files then aggregates realized interactive profit and realized NPV over the whole episode. The repository keeps the legacy column name `expected_profit_mean` in its exported summary tables, but the current `episode_summary()` implementation computes this value from realized weekly profits plus terminal settlement. In the discussion below, the original column name is retained when citing output files, but substantively it should be read as realized final portfolio profit under the simulator.

4.3 Scenario design

The six active evaluation scenarios are loaded from `configs/scenarios.yaml`. They are not arbitrary labels. Each one stresses a specific weakness of a threshold policy and therefore provides a reasoned test bed for cross-controller interpretation.

Scenario	Interpretation
`base_market`	Stable market with moderate approval and default dynamics. This scenario is the cleanest test of whether the controller can exploit signal without fighting strong regime shifts.
`drift`	Gradual deterioration in new-client scores, rising default pressure, mild volume growth, and a small negative recovery shift. This tests whether the controller reacts adequately to a slow but persistent quality decline.
`adverse_stress`	Abrupt negative shock, stronger default increases, higher noise, and larger loan amounts. This scenario punishes excessive risk-taking and highlights the value of robust selectivity.
`class_imbalance_shift`	Growing new-client share combined with lower average new-client quality. This scenario rewards controllers that manage composition risk rather than only overall approval.
`noise`	Stable mean structure with much noisier measurements and weekly application volume. This is a test of robustness against unstable observations and volume shocks.
`split_policy_dynamics`	New clients deteriorate while repeat clients improve. This is the most direct test of whether a split threshold is useful and whether segmented state information is exploited well.

Table 1: Scenario logic from `configs/scenarios.yaml`.

These scenarios are not only descriptive. They help explain why controller rankings differ. A myopic weekly profit-search baseline can dominate when the environment remains interpretable through current cohort economics, while a more state-aware RL controller can gain ground when segment asymmetry or stable market signal is exploitable. Under severe stress, however, strong risk control can matter more than flexible representation.

5 Controlled Experimental Design

5.1 What is held fixed

The present dimensionality experiment is intentionally controlled. The following elements remain constant across the 12D, 20D, 30D, and 50D runs.

- weekly simulator dynamics and the six-scenario evaluation suite;
- split-policy control semantics and the threshold grid from 35 to 85 with step 5;
- reward design, including delayed reward, risk-aware shaping, and constraint penalties;
- warm-up logic, interactive horizon, and terminal settlement;
- seeds 11, 23, and 37 in the active `quick` profile;
- training episodes, evaluation-run counts, and scenario randomization during training;
- the compared controller set: `dqn`, `double_dqn`, `ppo`, `sac`, `static_threshold`, `profit_oriented`, `risk_aware_weekly`, `constraint_aware_weekly`, and `split_policy_static`;
- the same bootstrap-CI machinery and the same output file structure.

The code even performs explicit repository-level validation to verify that the first 12 features are unchanged across every observation size, that controller coverage is complete, that required files exist, and that no future leakage is introduced by the added state coordinates.

5.2 What changes

The only experimental factor is the observation dimensionality:

$$\texttt{state_dim} \in \{12, 20, 30, 50\}.$$

The dimensionality comparison is therefore a study of state representation under constant environment and policy semantics. This is the correct framing for the thesis. The experiment is not testing different simulators, different rewards, or different datasets. It is testing whether additional state information improves the quality of threshold control and, if so, where the returns start to diminish.

5.3 Observation layers

The state space is expanded cumulatively.

State size	Composition
12D	Baseline state: week progress, overall and segment approval rates, rolling realized default rate, expected current default rate, realized profit, rolling profit volatility, projected capital usage, outstanding ratio, and normalized new/repeat thresholds.
20D	12D plus repeat-share, expected profit and expected NPV per application, realized NPV, weekly reward, threshold gap, capital headroom, and the gap between realized and expected default rate.
30D	20D plus lagged and rolling history: second-lag approval/default/profit/capital usage, four-week rolling approval/default/profit indicators, profit rolling standard deviation, and lagged threshold changes.
50D	30D plus richer segment and stability descriptors: segment-specific approval/default/profit signals, accepted-share ratios, rolling reward moments, cumulative reward and profit to date, capital-usage variability, outstanding-capital change, threshold-gap history, and current application-volume ratio.

Table 2: Cumulative observation construction. Exact ordered definitions are documented in `state_dimension_manifest.md`.

This cumulative construction gives the state-size experiment a substantive interpretation.

- The 12D state is essentially a compact weekly dashboard: what just happened, how risky it looks, and what thresholds were used.
- The 20D state adds immediate economic context. It tells the controller not only what happened, but how much profit and NPV the last cohort implied and how much capital room remains.
- The 30D state adds memory. It allows the controller to compare the latest week with lagged and rolling history, which is especially relevant under drift and noisy volume.
- The 50D state adds greater segmentation and stability detail. It is the richest description, but also the most demanding one for optimization and generalization.

From a methodological perspective, the central hypothesis is that the 20D and 30D enrichments add signal that is genuinely useful for portfolio control, whereas 50D may start to add complexity faster than control value. The results support exactly that intermediate-saturation interpretation.

6 Controllers

6.1 Rule-based and static baselines

The baseline set is economically meaningful rather than decorative.

- `static_threshold` applies one fixed shared threshold and therefore acts as the simplest non-adaptive benchmark.
- `split_policy_static` keeps different default thresholds for new and repeat clients but does not update them over time.

- `profit_oriented` performs weekly grid search and selects the threshold pair with the highest current-cohort expected profit, deliberately ignoring broader portfolio constraints.
- `risk_aware_weekly` is a hand-crafted feedback controller that raises or lowers thresholds in response to rolling realized defaults, approval pressure, and capital usage.
- `constraint_aware_weekly` performs a weekly grid search with penalty-weighted violation terms, thereby operationalizing a more conservative multi-objective benchmark.

These baselines are important for interpretation because they span several distinct management styles: static, split-static, myopic profit chasing, hand-built reactive control, and explicit constraint-aware search. When RL loses to one of them, the thesis can say *which kind of non-RL policy* remains competitive.

6.2 RL agents

The repository evaluates four RL algorithms.

- `dqn` and `double_dqn` operate on the discrete threshold grid and are natural choices when weekly control is represented as selection from a finite action set.
- `ppo` and `sac` use continuous actions that are mapped back onto the same threshold range and rounded to the grid granularity.

This separation is analytically useful because it shows that dimensionality interacts with action representation. The current quick-profile results do not produce a single RL story. Instead, they show that value-based discrete controllers benefit much more from enriched state than the two continuous-policy agents do under the same budget.

7 Reward Formulation and Metric Design

7.1 Reward function

The reward mechanism is implemented in `src/rl_credit_scoring_sim/env/reward.py`. With delayed reward enabled, the main terms are based on realized portfolio profit and realized NPV rather than only the current cohort's expectation. In simplified form, the weekly reward can be written as

$$\begin{aligned} r_t = w_p \cdot \Pi_t^{realized} + w_n \cdot \mathrm{NPV}_t^{realized} - w_s \cdot \hat{d}_t \cdot 100 \\ - \lambda_d \big[\max(0, \bar{d}_t - 0.12)\big]^2 \cdot 1000 \\ - \lambda_a \left|\alpha_t - 0.42\right| \cdot 1000 \\ - \lambda_c \big[\max(0, c_t - 0.82)\big]^2 \cdot 1000 \\ - \lambda_v \max\left(0, \frac{\sigma_{\Pi,t} - 8500}{8500}\right) \cdot 1000, \end{aligned} \tag{1}$$

where the active configuration sets $w_p = 1.0$, $w_n = 0.25$, $w_s = 0.35$, $\lambda_d = 9.0$, $\lambda_a = 2.0$, $\lambda_c = 3.5$, and $\lambda_v = 1.4$.

The reward is therefore intentionally multi-objective. It does not rank policies by raw profit alone. It rewards realized portfolio payoff, values discounted cash flow, discourages high expected default on the current accepted cohort, penalizes excess realized default in the rolling reward window, penalizes large deviations from the approval target, penalizes excessive projected capital usage, and penalizes excessive realized-profit volatility.

7.2 Why this trade-off matters

The reported results make little sense if the reader interprets them as a pure-profit contest. The repository's official selection rule for the best controller is: highest `cumulative_reward_mean`, then highest `expected_profit_mean`, then highest `stability_index_mean`, then lowest `default_rate_mean`. Consequently, a controller may produce slightly higher raw profit in one scenario and still lose on the main ranking if it does so with more volatility, stronger capital pressure, or more undesirable approval behavior.

The `noise` scenario is the clearest example. Across dimensions, `risk_aware_weekly` has the highest scenario-level profit by the repository's `expected_profit_mean` column, reaching 97,377.95 in that scenario. Yet the reward-based scenario winner remains `profit_oriented`, whose lower raw profit of 94,589.22 is offset by better total reward under the implemented weighting and penalty structure. This is not a contradiction; it is exactly what a shaped portfolio reward is supposed to reveal.

7.3 Reported metrics

The core metrics used throughout the outputs are:

- approval rate;
- realized default rate;
- final portfolio profit (exported under the legacy name `expected_profit_mean`);
- realized portfolio NPV;
- cumulative shaped reward;
- mean projected capital usage ratio;
- reward volatility;
- stability index, defined in code as $1/(1 + \text{reward_volatility})$;
- threshold volatility, defined as the standard deviation of week-to-week threshold changes inside an episode.

Note that threshold volatility measures within-episode movement, not cross-seed variation in the chosen threshold level. This distinction becomes important when interpreting the best RL runs, because the exported winners are usually static within an episode even though different seeds may settle on different threshold pairs.

8 Experimental Protocol

The main thesis evidence comes from the active `quick` profile. The merged configuration loaded by the code corresponds to:

- 8 warm-up weeks and 26 interactive weeks;
- 260 configured applications per week before scale factors;
- training scale 0.80, validation scale 0.90, and test scale 1.00 with `test_subset_fraction` = 0.65;
- 12 training episodes per RL seed;
- 4 evaluation runs per seed and scenario;
- seeds 11, 23, and 37;
- bootstrap confidence intervals at the 95% level with 120 resamples in the active fast-CI mode because `full_ci = false`.

The outputs are organized as follows.

- One directory per observation size: `outputs/dim_12/`, `outputs/dim_20/`, `outputs/dim_30/`, and `outputs/dim_50/`.
- Root-level cross-dimension summaries: `outputs/dimension_comparison_summary.csv`, `outputs/best_r` `outputs/overall_best_by_dimension.csv`, and the `metric_vs_dimension_*.png` figures.

The controlled-protocol validation is one of the stronger aspects of the current repository state. The dimensionality pipeline checks that:

- the first 12 features are unchanged across all dimensions under a common reset and fixed action sequence;
- seeds, scenarios, reward settings, and controller sets match across dimensions;
- the added features use only interactive history, last-week metrics, week index, and fixed configuration values;
- every controller finishes the expected number of runs in every dimension;
- all required CSV and plot artifacts exist and are non-empty;
- the best-row extracts are consistent with the global comparison summary.

This validity layer is valuable for the thesis because it supports the claim that the reported saturation pattern is not an artifact of silent protocol drift.

9 Main Results

9.1 Best overall controller by dimension

Table 3 reports the strongest controller when baselines and RL agents are pooled together under the repository's official reward-based ranking rule.

State dim	Best overall	Type	Profit	Reward	Default rate
12	`profit_oriented`	baseline	76 022.08	88 080.60	0.1169
20	`profit_oriented`	baseline	76 022.08	88 080.60	0.1169
30	`profit_oriented`	baseline	76 022.08	88 080.60	0.1169
50	`profit_oriented`	baseline	76 022.08	88 080.60	0.1169

Table 3: Best overall controller by dimension from `outputs/overall_best_by_dimension.csv`.

Three points are important here. First, the flat baseline line across dimensions confirms that the experiment is truly isolating the effect of observation size on RL controllers rather than on the comparison set. Second, the leading baseline is not overwhelmingly better than every other non-RL policy. The runner-up `constraint_aware_weekly` achieves cumulative reward 88,000.75, only about 79.85 below `profit_oriented`. Third, this means the top of the leaderboard is dominated by strong rule-based benchmarks, not by a trivial static threshold.

In substantive terms, the best overall result says that the current quick-profile environment still rewards well-designed weekly rule search. RL narrows the gap as the state becomes richer, but the repository does not support the stronger claim that RL already dominates policy search under all tested conditions.

9.2 Best RL controller by dimension

State dim	Best RL	Profit	NPV	Reward	Approval rate	Capital usage
12	`dqn`	59 197.47	57 144.61	67 308.36	0.4159	0.4454
20	`double_dqn`	61 096.44	58 978.98	69 812.02	0.4329	0.4569
30	`dqn`	66 758.29	64 755.07	78 219.69	0.3527	0.3902
50	`dqn`	66 946.81	64 815.05	78 198.39	0.3935	0.4280

Table 4: Best RL controller by dimension from `outputs/best_rl_by_dimension.csv`.

Table 4 gives the main empirical pattern of the study.

1. The move from 12D to 20D helps modestly. The best RL profit rises from 59,197.47 to 61,096.44, and cumulative reward rises from 67,308.36 to 69,812.02.

2. The move from 20D to 30D is the decisive improvement. Profit increases by about 5,661.85, NPV increases by about 5,776.10, cumulative reward increases by about 8,407.67, while default rate declines from 0.1260 to 0.1185.

3. The move from 30D to 50D yields almost no additional reward. Profit improves by only about 188.53, cumulative reward is slightly lower, and both approval rate and capital usage rise again.

This pattern is exactly what a saturation story looks like. The 20D and 30D states add information that can be monetized into better control. The 50D state does not extend that gain in a comparable way.

The 30D row is especially interesting because it improves reward while becoming *more selective*. Approval rate falls to 0.3527 and capital usage falls to 0.3902. This is not a reward gain caused by simply approving more loans. It is a gain produced by accepting a cleaner and more capital-efficient portfolio mix. That strengthens the interpretation that the additional lagged and rolling state variables are economically useful rather than merely descriptive.

9.3 Algorithm-specific interaction with dimensionality

The aggregate best-RL frontier hides important heterogeneity across algorithms. Table 5 reports all four RL agents.

Agent	State dim	Profit	Reward	Default rate	Threshold volatility
`dqn`	12	59 197.47	67 308.36	0.1233	0.0000
`dqn`	20	-15 413.93	-34 719.92	0.1545	0.0000
`dqn`	30	66 758.29	78 219.69	0.1185	0.0000
`dqn`	50	66 946.81	78 198.39	0.1195	0.0000
`double_dqn`	12	59 221.98	66 915.54	0.1223	0.0000
`double_dqn`	20	61 096.44	69 812.02	0.1260	0.0000
`double_dqn`	30	57 467.87	65 247.13	0.1282	0.0000
`double_dqn`	50	-49 782.91	-79 329.16	0.1574	0.0000
`ppo`	12–50	12 196.74	4 958.59	0.1537	0.0000
`sac`	12	-36 379.87	-57 961.65	0.1674	3.0132
`sac`	20	-22 232.63	-35 997.61	0.1494	2.2454
`sac`	30	-11 904.25	-26 218.31	0.1535	2.6183
`sac`	50	-72 433.15	-105 961.13	0.1677	1.9269

Table 5: All RL agents across dimensions from the per-dimension `main_overall_summary.csv` files.

The algorithm-specific picture matters as much as the best-RL picture.

- DQN is highly sensitive to representation quality. It performs well at 12D, fails badly at 20D, then becomes the best RL controller by a wide margin at 30D and stays there at 50D.
- Double-DQN peaks earlier. It is the best RL method at 20D, remains competitive at 30D, and then collapses at 50D.
- PPO is numerically identical across all four dimensions down to the exported summary precision. Under the current quick-profile training budget, richer state does not change its learned outcome materially.

- SAC remains negative in every dimension and becomes especially poor at 50D.

This is strong evidence that dimensionality is not an isolated "more is better" dial. It interacts with optimization method and training budget. The added state dimensions are useful enough to improve the best RL frontier, but they do not produce universal improvement across all controllers.

9.4 Cross-dimension figures

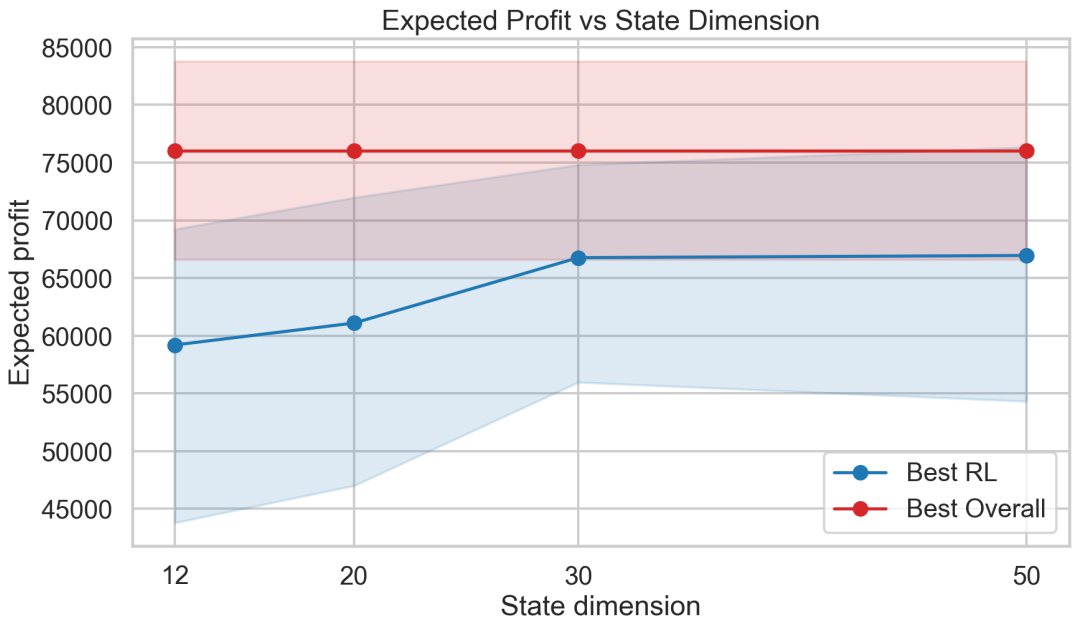


Figure 1: Profit versus state dimension for the best RL controller and the best overall controller.

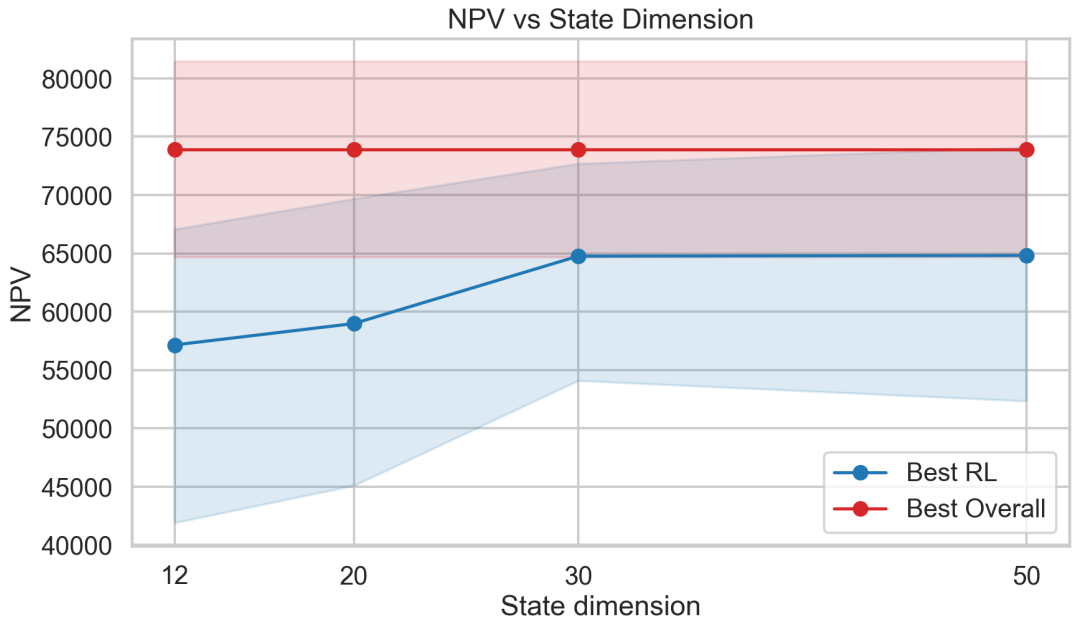


Figure 2: NPV versus state dimension.

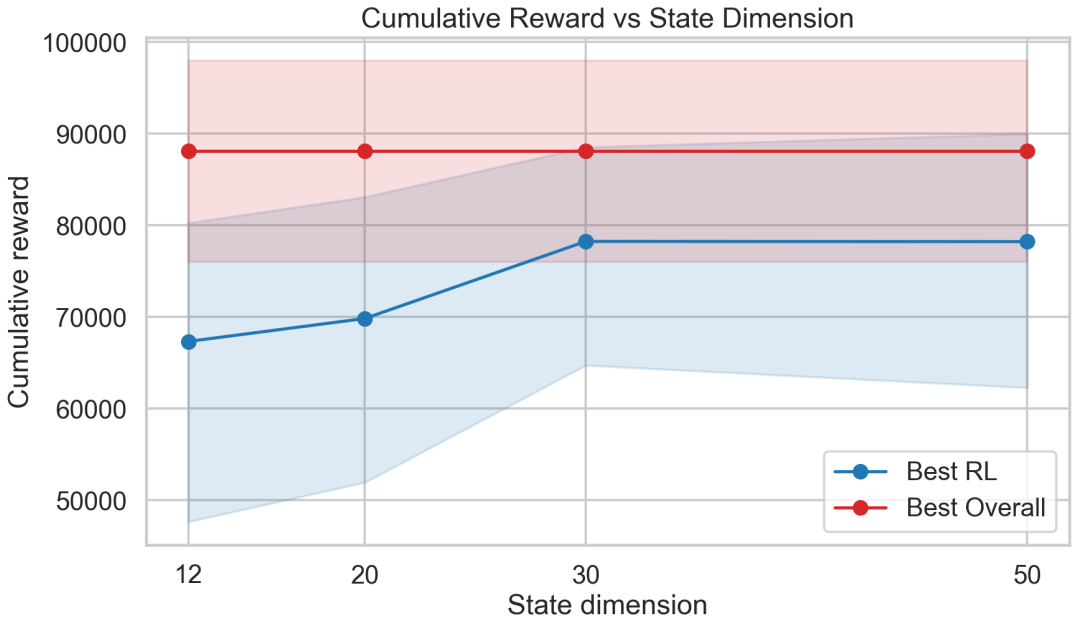


Figure 3: Cumulative reward versus state dimension.

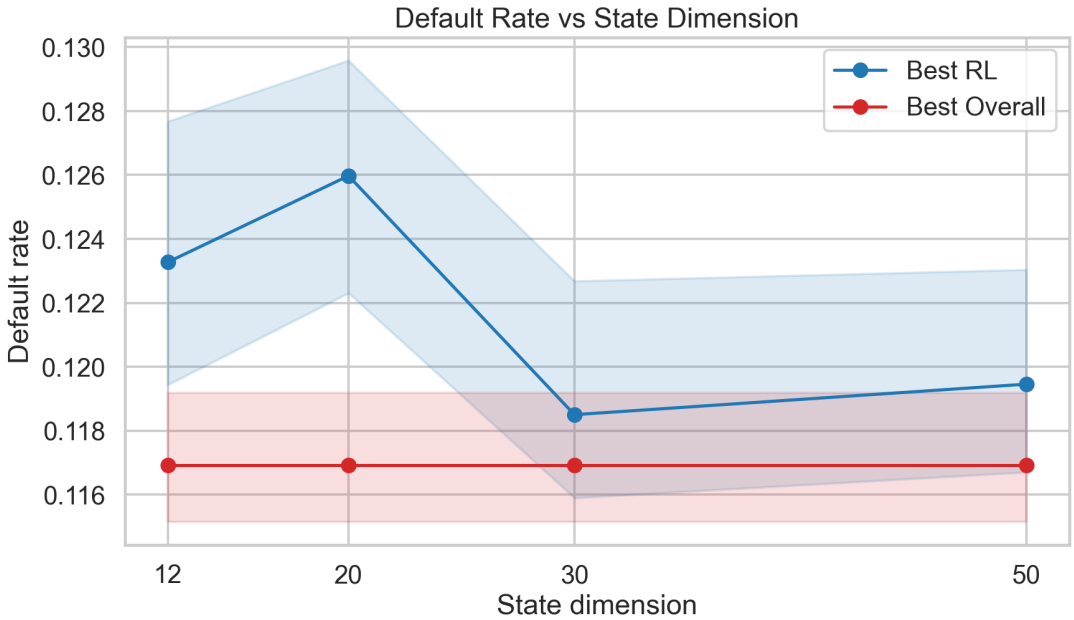


Figure 4: Default rate versus state dimension.

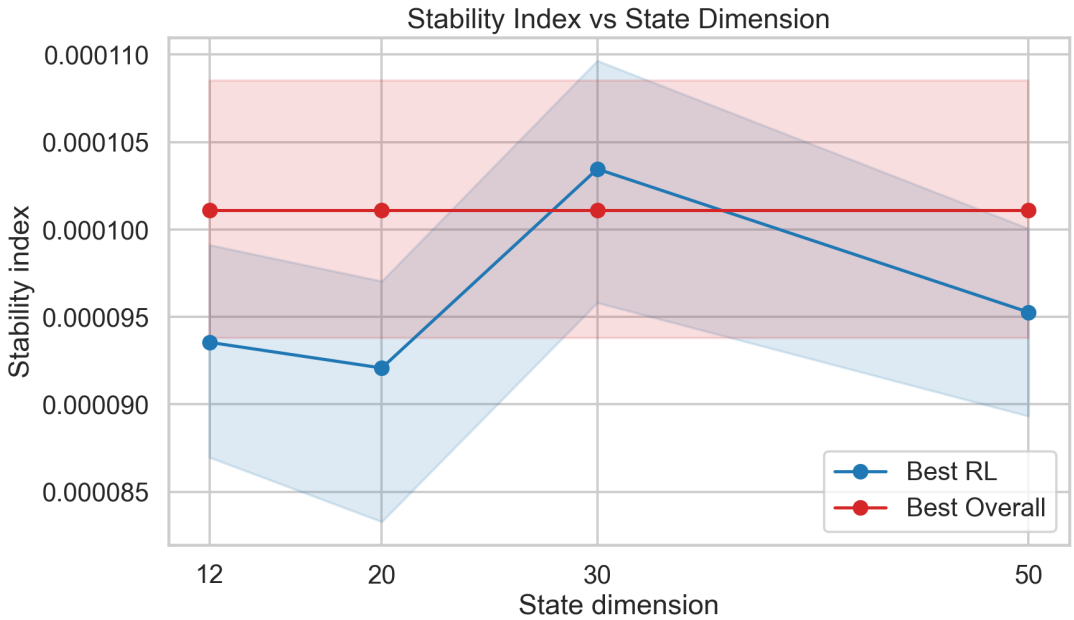


Figure 5: Stability index versus state dimension.

Figures 1–5 summarize the same story visually. The best-overall line is flat because the best baseline does not depend on the varying state vector. The best-RL line rises from 12D to 30D on profit, NPV, and reward, then flattens. Default rate improves sharply at 30D, then drifts slightly upward at 50D. Stability changes little, which means the main gain is not a large volatility reduction but a better balance of selectivity, realized performance, and capital usage.

10 Scenario-Level Results and Interpretation

10.1 Why scenario-level analysis is necessary

The overall leaderboard is informative but incomplete. Because the reward aggregates across six scenarios, it can hide the situations in which RL already matches or exceeds the best non-RL policy and the situations in which it still falls far short. The repository exports scenario-level summaries precisely to avoid that loss of structure.

Two different notions of "winner" must be distinguished.

- The *reward-based winner* is the controller selected by the repository's official ranking logic.
- The *highest-profit winner* is the controller with the largest `expected_profit_mean` in the output tables.

These are not always the same controller because the reward function is intentionally multi-objective.

10.2 Scenario trade-offs in the richest state

Table 6 focuses on 50D because it is the richest observation design and makes the reward-profit-risk distinction especially visible.

Scenario	Reward-based winner	Highest-profit winner	Lowest-default winner
adverse_stress	profit_oriented	profit_oriented	profit_oriented
base_market	dqn	dqn	dqn
class_imbalance_shift	constraint_aware_weekly	constraint_aware_weekly	dqn
drift	profit_oriented	profit_oriented	profit_oriented
noise	profit_oriented	risk_aware_weekly	dqn
split_policy_dynamics	profit_oriented	profit_oriented	dqn

Table 6: Reward-profit-risk leaders in the 50D run, computed from `outputs/dim_50/main_scenario_summary.csv`.

This table shows why the repository cannot be summarized by one sentence such as "baseline wins" or "RL wins." The answer depends on the metric and the scenario.

10.3 Adverse stress

Adverse stress remains the hardest scenario for RL in every observation size. The reward-based winner is always `profit_oriented`, with profit 29,336.42, reward 25,150.74, and default rate 0.1178. The best RL result is much weaker: in 12D, the best RL controller is `double_dqn` with negative profit; in 30D, DQN recovers to a small positive profit of 3,413.22 but still trails the baseline heavily; in 50D, DQN falls back to −12,023.13.

This pattern is plausible given the scenario design. Adverse stress combines a shock, stronger default pressure, higher noise, and larger loan amounts. Under those conditions, the cost of over-acceptance rises sharply. The myopic-looking `profit_oriented` baseline is not actually reckless here because the current-cohort profit preview already reflects much worse cohort economics when the market deteriorates. Its weekly grid search therefore reacts by becoming highly selective, and that conservatism is rewarded.

The cross-week snapshots reinforce the same interpretation. In 30D adverse stress, DQN improves substantially over its 12D and 20D versions, but at week 20 cumulative profit remains only about 7,482.68 compared with `profit_oriented`'s 22,046.88. Richer state helps the RL policy survive stress more gracefully, yet it does not close the gap to the best baseline within the current quick-profile budget.

10.4 Base market

Base market is where RL looks strongest. In 12D, DQN is already the reward-based winner with profit 86,519.38 and reward 101,967.98. In 20D, Double-DQN becomes the scenario winner and pushes profit to 94,321.51. In 30D, Double-DQN still leads the scenario on reward, while in 50D DQN becomes the clear leader with profit 92,105.55, reward 110,521.41, and default rate 0.1134.

This result is economically sensible. Base market offers the cleanest environment for representation value to matter. The controller does not have to fight a regime break as severe as adverse stress, so richer state can be used to set a more efficient selective policy. The 50D DQN result is notable because it exceeds the `profit_oriented` baseline not only on reward but also on raw

profit and default rate. That is the clearest case in the exported results where RL converts richer portfolio state into both higher return and better credit quality.

At the same time, the base-market result should not be over-generalized. The winning RL agent changes across dimensions, and Double-DQN collapses at 50D overall despite once leading this scenario. This again shows that scenario-level success and global robustness are not the same thing.

10.5 Class-imbalance shift

Class-imbalance shift is the most consistent win for the conservative rule-based benchmark. The reward-based and profit-based winner is always `constraint_aware_weekly`, with profit 70,649.29 and reward 81,420.58. This makes intuitive sense because the scenario increases the share of lower-quality new applicants. A controller that explicitly internalizes approval, default, and capital penalties has a structural advantage.

The RL story is still important, however. The best RL profit rises from 56,746.75 at 12D to 65,715.59 at 50D, and the lowest default rate in 30D and 50D belongs to DQN rather than to the constraint-aware baseline. This indicates that richer state helps RL recognize composition risk and respond more selectively. What it does not yet do in the quick profile is beat the explicit rule search that is already well aligned with the scenario.

10.6 Drift

In the drift scenario, `profit_oriented` is the reward-based winner throughout the dimensionality sweep, with profit 59,526.10 and reward 68,132.12. The best RL policy improves from around 40,685.98 in 12D to 52,005.69 in 30D, then softens to 46,567.72 in 50D.

This is a useful counterexample to a naive "more dimensions always help" claim. The 30D state, which adds lagged and rolling history, is especially well suited to slow deterioration because drift is a temporal phenomenon. The 50D state adds more descriptors, but those extra coordinates do not improve DQN further in this scenario. The likely implication is that the 30D historical layer captures the essential signal, while the 50D enlargement adds modeling burden without equivalent incremental value.

10.7 Noise

The noise scenario reveals the difference between profit maximization and shaped reward most clearly. The highest raw profit belongs to `risk_aware_weekly` at 97,377.95, while the reward-based winner remains `profit_oriented` at profit 94,589.22 and reward 112,586.40. In 30D and 50D, DQN achieves lower default rates than both of those baselines and still posts very strong profits, namely 88,596.00 in 30D and 89,396.58 in 50D.

The interpretation is nuanced. Under noisy weekly volumes and measurement disturbance, the risk-aware baseline appears to exploit current profit well, but the reward function still values the more balanced profile produced by `profit_oriented`. At the same time, richer-state DQN becomes meaningfully safer, with default rates around 0.1133 in 30D and 0.1160 in 50D, compared with `risk_aware_weekly`'s 0.1222. This is one of the best examples of portfolio-quality trade-off

in the current outputs: RL does not win the main ranking, but it creates a more conservative portfolio without collapsing profit.

10.8 Split-policy dynamics

The split-policy scenario is designed to reward controllers that can benefit from separate thresholds. The strongest reward-based controller remains `profit_oriented` across all dimensions, but the gap to the best RL agent shrinks sharply as state richness increases. The best RL profits are 114,185.42 at 12D, 117,912.12 at 20D, 112,439.04 at 30D, and 119,918.57 at 50D.

The most important observation is not only that 50D DQN reaches almost the same profit as the baseline. It also achieves the lowest default rate in the 50D scenario summary at approximately 0.1093, compared with `profit_oriented`'s 0.1112. This is exactly the kind of pattern the richer segmented state was meant to uncover: when new and repeat segments evolve in opposite directions, a controller with better segment-specific state can approach baseline-level profitability while becoming slightly safer.

11 Threshold Behavior and Dynamic Response

11.1 What threshold volatility actually shows

At first glance, the main summaries may suggest that the best RL policies are extremely stable because their reported `threshold_volatility_mean` is exactly zero. That interpretation is correct, but only in a specific sense. Threshold volatility is defined from week-to-week changes within an episode. A value of zero therefore means that the policy keeps the same threshold pair throughout the episode.

The exported weekly logs confirm that this is exactly what happens in the quick-profile winners. For the best RL controller in each dimension, 100% of evaluation episodes are constant-threshold episodes. The improvement from 12D to 30D does not come from more active week-to-week motion. It comes from selecting better threshold pairs.

11.2 Episode-static RL winners

The best RL controllers in the quick profile choose only a small set of episode-level threshold pairs:

- 12D DQN selects the pairs $(65, 45)$, $(85, 35)$, and $(85, 50)$ across episodes.
- 20D Double-DQN selects $(65, 40)$, $(75, 50)$, and $(80, 55)$.
- 30D DQN selects $(70, 65)$, $(85, 35)$, and $(85, 50)$.
- 50D DQN selects $(70, 45)$, $(85, 40)$, and $(85, 45)$.

This fact is important for honest interpretation. In the active quick-profile experiment, richer state helps the controller identify a better static split policy for the episode more than it induces genuinely adaptive weekly threshold movement. That does not invalidate the dimensionality result.

It simply narrows its meaning. The current evidence supports "better state improves policy quality" more strongly than "better state produces richer within-episode adaptation."

This also helps explain why the leading dynamic baselines still perform so well. Their threshold-volatility means are large—about 9.83 for `profit_oriented` and 9.60 for `constraint_aware_weekly` in the overall summaries—because they reoptimize or adjust thresholds every week. The quick-profile RL winners, by contrast, are strong mostly as episode-level selectors of static threshold pairs.

11.3 Early-loss / late-gain diagnostic

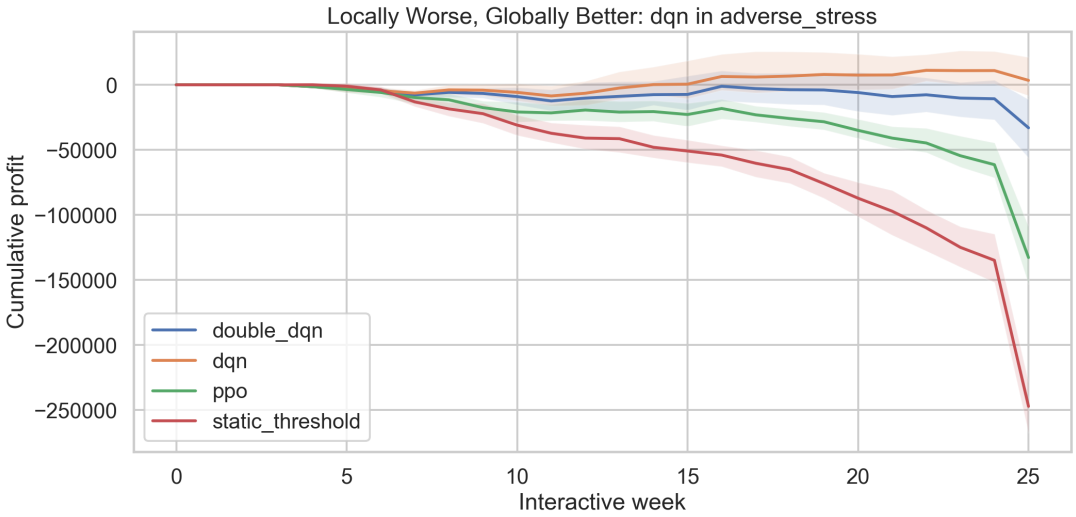
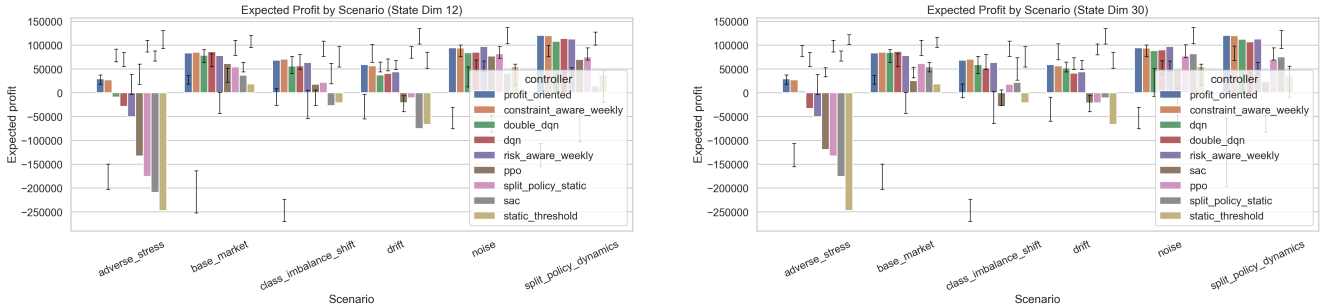


Figure 6: Diagnostic example of a locally worse but globally better trajectory from the 30D output folder.

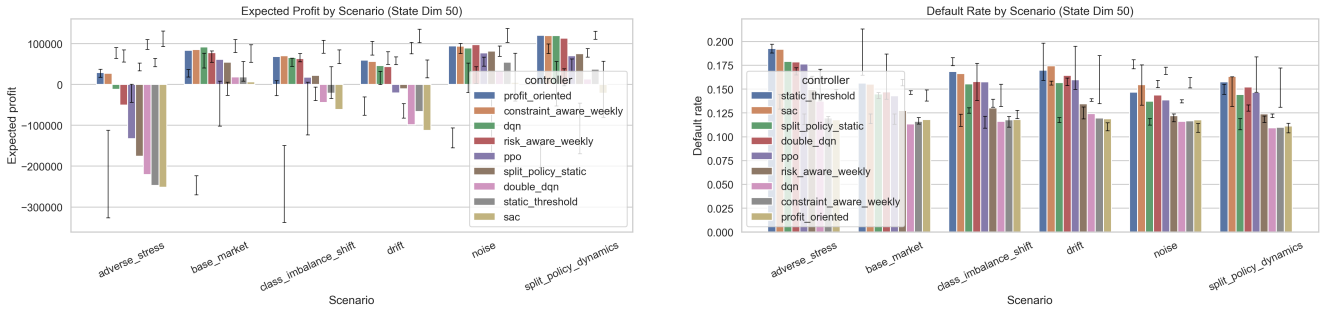
Figure 6 is useful conceptually even though it is only a diagnostic. It illustrates why delayed cash-flow accounting matters for thesis interpretation. A policy can fall behind early because stricter acceptance reduces immediate cohort contribution, then recover strongly once weaker loans that would have defaulted are never originated. That temporal reversal is exactly the type of behavior that a weekly delayed-outcome simulator is designed to capture and that static one-step classification metrics would miss.

12 Figures for Dimension Comparison



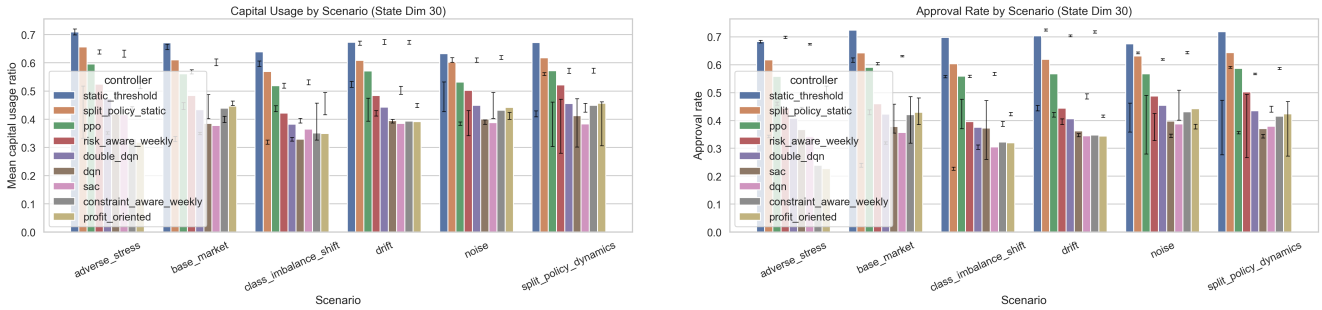
(a) 12D (b) 30D

Figure 7: Scenario-level profit comparison for two representative state sizes.



(a) 50D (b) 50D default-rate comparison

Figure 8: Rich-state scenario view of profit and default trade-offs.



(a) 30D capital usage (b) 30D approval rate

Figure 9: Portfolio-allocation trade-offs for the 30D run.

Taken together, Figures 7–9 show why the 30D and 50D results are best understood through scenario structure rather than through one scalar curve only. The richest state does help in several places, especially `base_market`, `class_imbalance_shift`, and `split_policy_dynamics`, but it does not remove the RL deficit under `adverse_stress` and does not overturn the dominance of conservative rule-based search in `drift`.

13 Discussion

13.1 Why 30D is the most balanced state size

The 30D state is the strongest compromise in the current outputs. It delivers the largest jump in best-RL reward, improves profit and NPV materially, lowers default rate relative to 20D, and does so with lower approval and capital usage. In other words, it improves the quality of selectivity rather than only the quantity of lending.

The extra information added between 20D and 30D is not arbitrary. It is precisely the historical memory layer: lagged approval, lagged realized default, lagged realized profit, lagged capital usage, rolling means, rolling profit variability, and lagged threshold changes. Those variables are well matched to a weekly control problem because they tell the controller whether the latest week is exceptional or part of an emerging pattern. That is especially useful in drift, noise, and composition-shift environments.

13.2 Why 50D saturates

The move from 30D to 50D still adds meaningful descriptors, but their incremental value is smaller. Segment-level means, rolling reward moments, cumulative totals to date, and threshold-gap history make the observation richer, yet the best RL reward no longer improves. In practice, this looks like a saturation frontier: once the controller already knows current performance, recent history, and lagged threshold behavior, the additional segmentation and stability coordinates do not produce a comparable gain in decision quality under the current training budget.

There is also a more cautionary reading. The 50D run is not merely "no better." It actively harms some algorithms. Double-DQN falls from a positive reward of 69,812.02 at 20D to −79,329.16 at 50D, and SAC deteriorates sharply as well. This means that the 50D state can be viewed as both richer and more optimization-demanding. The state itself is not bad, but its control value is conditional on the learning algorithm and the training budget.

13.3 Why the best baseline still wins overall

The persistent dominance of `profit_oriented` and `constraint_aware_weekly` is not accidental. Both baselines exploit privileged structural alignment with the environment.

- They perform explicit weekly search over the same threshold grid that the environment exposes.
- They do not have to learn a value function or policy representation from limited episodes.
- Their decisions are directly tied to the simulator's current weekly economics and constraint structure.

Under the quick profile, RL has only 12 training episodes per seed, which is deliberately small. The fair interpretation is therefore not "RL failed." The fair interpretation is that strong hand-crafted policy search remains extremely competitive in a well-structured simulator, while RL becomes meaningfully better when the state is informative enough and the scenario is favorable enough.

13.4 Portfolio-quality trade-offs

The dimensionality experiment is most interesting where several metrics move in opposite directions. The best RL controller at 20D increases approval rate and capital usage relative to 12D, but also slightly worsens default rate. The 30D controller reverses that pattern: it becomes more selective, uses less capital, and improves reward strongly. The 50D controller then relaxes again, with higher approval and capital usage but nearly flat reward.

This sequence is exactly the kind of portfolio trade-off that a thesis should emphasize. Better results do not always mean "approve more" or "reduce default" in isolation. What matters is the joint movement of profit, default, capital usage, and stability. The 30D result is strong because the joint movement is favorable. The 50D result is weaker because the additional approval and capital consumption do not translate into additional reward.

13.5 What the current results do and do not prove

The current evidence does prove several things.

- State design matters materially for RL quality.
- The marginal value of additional state information is non-linear.
- Richer state can improve profitability while preserving or even improving risk metrics in some scenarios.
- The benefit of richer state is algorithm-dependent.

It does *not* prove that 50D is universally too large, that 30D would remain optimal under a much larger training budget, or that RL is incapable of learning genuinely dynamic weekly policies. Those stronger claims would require a longer-horizon training study, not only the current quick-profile outputs.

14 Limitations

Several limitations should be stated directly.

- The simulator is synthetic. Its segment parameters and scenarios are economically plausible, but they are not calibrated to confidential production portfolios.
- The quick profile is intentionally small. Twelve training episodes per seed are sufficient for an end-to-end controlled experiment, not for a final convergence claim.
- The reward is a designed objective, not a discovered one. The relative ranking of controllers is therefore partly a function of the chosen trade-off between profitability, approval pressure, capital usage, and volatility.
- The exported confidence intervals quantify uncertainty inside the simulation generator. They do not, by themselves, establish external validity on real bank data.

- The best RL policies in this profile are episode-static threshold pairs. Therefore, the current evidence for adaptive weekly threshold movement is weaker than the evidence for better episode-level threshold selection.

These limitations do not weaken the core methodological contribution. They simply define the correct boundary of the claim.

15 Novelty and Contribution

The novelty of the study lies less in inventing a brand-new RL algorithm and more in combining several research ingredients in one controlled experimental design.

- The project formulates credit-scoring control as weekly portfolio threshold management rather than one-shot application classification.
- It makes delayed outcomes first-class objects of the environment, including late recovery and terminal settlement.
- It models new and repeat applicants separately and allows simultaneous control of both thresholds.
- It evaluates learned policies against a diverse and economically interpretable baseline set.
- It turns state design itself into a controlled research variable and documents the resulting saturation effect.

This is why the thesis contribution should be described as a contribution in *problem formulation, simulation design, and experimental methodology*, not only as a leaderboard comparison.

16 Conclusion

The current repository supports a clear and evidence-based conclusion. Increasing observation dimensionality from 12 to 20 and then to 30 features improves the best RL frontier substantially. The largest gain occurs at 30D, where DQN reaches profit 66,758.29, NPV 64,755.07, cumulative reward 78,219.69, and a reduced default rate of 0.1185 with lower approval and capital usage than the 20D winner. Moving from 30D to 50D adds almost no additional reward and slightly weakens the portfolio balance, which is the signature of saturation.

At the same time, the thesis should not compress the results into a simplistic RL-versus-baseline slogan. The strongest overall controller remains the rule-based `profit_oriented` baseline, with `constraint_aware_weekly` essentially tied behind it. RL is already competitive and occasionally dominant in some scenarios, especially stable or richly segmented ones, but it is not yet the universal winner under the current quick-profile budget.

The most defensible final statement is therefore the following. In this weekly delayed-outcome credit-scoring simulator, state representation is a first-order determinant of RL quality. Richer state adds real control value up to an intermediate regime, especially once lagged and rolling portfolio history are incorporated. Beyond that point, the marginal gain flattens and can even reverse for some algorithms. For the exported quick-profile results, 30D is the most balanced RL

state design, 50D demonstrates diminishing returns, and strong rule-based baselines remain the benchmark that RL must still surpass more consistently.